

Spęczator 2000

Szymon Miłkowski

12.01.2026

1 Cel i wstępne założenia

Celem tego projektu było stworzenie graficznego programu komputerowego do analizy, korekcji i transformacji danych pomiarowych z prób spęczania. Danymi wejściowymi do programu miały być pliki binarne w formacie *W01* generowane przez oprogramowanie maszyny badawczej.

Dodatkowym celem miała być kompatybilność i płynność działania zarówno na współczesnym sprzęcie, jak i starszym (Windows XP 32-bit).

2 Wybór technologii

Z racji na wymóg szerokiej kompatybilności, oraz wysokiej wydajności, konieczne było wybranie technologii pozwalającej na kompilację do natywnego kodu maszynowego z minimalną ilością zależności. Wybór padł na język C z biblioteką Raylib do obsługi grafiki, linkowaną statycznie.

Raylib posiada łatwe w obsłudze API składające się z funkcji do rysowania i obsługi wejścia (mysz, klawiatura).

3 Spęczanie

Próba spęczania polega na ściskaniu próbki wzdłuż jednej osi (pionowej) przez maszynę badawczą z zadaną prędkością. Podczas próby materiał odkształca się, zmniejszając swoją wysokość w kierunku siły oraz - zgodnie z zasadą zachowania objętości - zwiększając swoją szerokość w pozostałych kierunkach.

Podczas tego procesu, materiał wewnątrz płynie, wypychając materiał na ścianach bocznych na zewnątrz, a materiał przy podstawach trze o górną i dolną ścianę maszyny, co hamuje jego przemieszczenie na zewnątrz.

Siła nacisku (opór jaki stawia próbka) jest mierzona i zależy od materiału i fazy procesu.

3.1 Fazy procesu

Brak kontaktu

- Brak kontaktu - maszyna nie ściska próbki, zmierza w jej kierunku - stała, niska siła
- Pierwszy kontakt - luzy w maszynie są ściskane - szum informacyjny
- Faza liniowa?? - siła rośnie liniowo
- Faza nieliniowa?? - siła rośnie coraz mniej

4 Format pliku wejściowego

4.1 Nieznany format binarny

Pliki wejściowe z danymi pomiarowymi z rozszerzeniem *.W01*, zapisane są w nieudokumentowanym formacie binarnym. Wpisywanie w wyszukiwarce *.W01* lub magic number z pliku - pierwsze cztery bajty *66 66 86 3f* nie daje istotnych rezultatów. Aby zczytać dane z plików i stworzyć algorytm importujący do programu, należy więc przeprowadzić inżynierię wsteczną formatu.

4.2 Inżynieria wsteczna formatu

W inżynierii wstecznej nie ma jednego poprawnego przepisu na znalezienie rozwiązania. Każde zadanie jest osobną łamigłówką, do której musimy użyć narzędzi, doświadczenia, wiedzy informatycznej, wiedzy dziedzinowej oraz czasu. Podczas analizy od dyspozycji miałem dwa pliki: *19.v10.W01* oraz *20.v50.W01*, gdzie *v10* i *v50* odpowiadają prędkości procesu spęczania.

4.2.1 Pierwsze spojrzenie na plik

Rzut okiem w *hex-edytorze* ujawnia kilka stringów i sporo zer na początku pliku, a w dalszej części powtarzający się czterokrotnie wzór: napis i blok nieczytelnych danych.

4.2.2 Nagłówek

W nagłówku pliku, który znajduje się na początku, przed właściwymi danymi liczbowymi znajdują się metadane, informacje o rozmiarze, godzina utworzenia oraz nazwa programu wejściowego do maszyny badawczej. Cały nagłówek ma stały rozmiar 208 bajtów.

Struktura nagłówku pliku				
offset	rozmiar	wartość/typ	nazwa	komentarz
0	4	0x3f866666	magic	stała wartość określająca format
4	4	1	wersja?	najpewniej wersja formatu
8	4	$\text{rozmiar_pliku} + 1$	rozmiar	
12	4	0	nieznana	zawsze 0
16	8	"ScriptPl"	stały napis	8 znaków, bez null terminatora
24	4	$\text{rozmiar_pliku} - 16$	rozmiar2	wygląda jak zbędna informacja, może oznaczać "rozmiar począwszy od napisu"
28	4	liczba wierszy	liczba wierszy	ilość liczb w każdej kolumnie danych
32	4	liczba kolumn	liczba kolumn	ilość kolumn z danymi
36	4	1	nieznane	
40	4	1	nieznane	
44	18	"dd-MM-yyyyhh:mm:ss"	czas utworzenia	napis bez null terminatora
62	42	nieznana	nieznane	42 bajty, głównie zera
104	104	nazwa pliku	plik badania	nazwa pliku wejściowego do maszyny, dopełniona zerami do 104 bajtów

4.2.3 Kolumny (serie danych)

Kolumny mają zmienny rozmiar, zależnie od pliku, jednak zawsze taki sam w obrębie jednego pliku.

Struktura kolumny				
offset	rozmiar	wartość/typ	nazwa	komentarz
0	4	int	nieznane	1 lub 16385
4	59	napis	nazwa	nazwa i jednostka kolumny, oddzielone spacjami
63	$4 * \text{liczba_wierszy}$	float	dane	tablica 32-bitowych liczb zmiennoprzecinkowych

4.2.4 Struktura całego pliku

Plik składa się z nagłówka o stałym rozmiarze i zmiennej liczby kolumn zmiennej długości, które zawierają swój nagłówek oraz danych liczbowych.

Struktura pliku	
rozmiar	element
208	nagłówek
$63 + 4 * liczba_wierszy$	kolumna 1
$63 + 4 * liczba_wierszy$	kolumna 2
...	...
$63 + 4 * liczba_wierszy$	kolumna n

5 Implementacja

5.1 Ogólna struktura aplikacji

5.1.1 Komponenty

Interfejs graficzny aplikacji podzielony jest na "komponenty" - funkcje zawierające logikę kontrolną oraz rysujące pewien element interfejsu. Wywołanie tej funkcji oznacza, że komponent istnieje w obecnej klatce. Funkcje te mogą wywoływać inne funkcje komponentów, co tworzy swego rodzaju strukturę drzewa komponentów, rozpoczynającą się w głównej pętli programu. Pozycja w strukturze drzewa nie ma jednak wpływu na rozmieszczenie elementu, pozycja jest statyczna, lub określana poprzez podane parametry.

5.1.2 Zarządzanie stanem

Inspirowane działaniem innego drzewiastego systemu komponentów graficznych - webowym Reactem - komponenty, jeśli to konieczne, operują na dwóch rodzajach danych: właściwości (*props*) i stan (*state*).

Właściwości są podawane do funkcji komponentu jako kopia (pass by value), bądź stały wskaźnik, mają za zadanie sterować zachowaniem i wyglądem komponentu.

Stan jest podawany jako wskaźnik do pamięci zarządzanej przez rodzica (komponent wywołujący), ma za zadanie przechowywanie stanu komponentu między wywołaniami, jest modyfikowany głównie przez sam komponent oraz jego dzieci (jeśli zostanie podany do nich jako ich stan).

Stan globalny jest wykorzystywany w niektórych komponentach, które z założenia mają istnieć tylko w jednej sztuce jednocześnie, jak lista otwartych plików lub narzędzia diagnostyczne *FPSCounter* i *Clock*, jako zmienne globalne w jednostce translacyjnej (niewidoczno poza obecnym plikiem .c), lub statyczne zmienne lokalne w funkcji komponentu.

5.2 Rysowanie wykresu

Raylib zawiera jedynie proste funkcje rysowania kształtów. Jedną z nich jest *DrawSplineLinear*, łącząca podane współrzędne pikseli liniowymi odcinkami. cośtam właśnie jej użyjemy bla

```
DrawSplineLinear(state->point_buffer, state->visible.count, 2.f, DARKBLUE);
```

5.2.1 Poruszanie się w widoku

Aby zwiększyć interaktywność, użytkownik może się poruszać po płaszczyźnie XY, przeciągając płaszczyzną myszką, oraz przybliżając i oddalając kółkiem myszy. Manipulowane są w ten sposób zmienne skali oraz przesunięcia uwzględnione w przekształceniu punktów ze współrzędnych kartezjańskich na współrzędne ekranu.

5.2.2 Transformacja współrzędnych

Punkt podlega przekształceniu liniowemu:

$$x = \frac{z(x_p - x_v) - x_S}{x_E - x_S} * W + x_O \quad (1)$$

gdzie

z - współczynnik skali

x_p - współrzędna x punktu w układzie kartezjańskim

x_v - przesunięcie x w układzie kartezjańskim

x_S - współrzędna x lewej krawędzi układu kartezjańskiego przy zerowym przesunięciu

x_E - współrzędna x prawej krawędzi układu kartezjańskiego przy zerowym przesunięciu

W - szerokość wykresu w pikselach

x_O - odległość prawej krawędzi wykresu od prawej krawędzi ekranu

$$y = \frac{-z(y_p + y_v) - y_S}{y_E - y_S} * H + y_O \quad (2)$$

gdzie

z - współczynnik skali

y_p - współrzędna y punktu w układzie kartezjańskim

y_v - przesunięcie y w układzie kartezjańskim

y_S - współrzędna y dolnej krawędzi układu kartezjańskiego przy zerowym przesunięciu

y_E - współrzędna y górnej krawędzi układu kartezjańskiego przy zerowym przesunięciu

H - wysokość wykresu w pikselach

y_O - odległość górnej krawędzi wykresu od górnej krawędzi ekranu

Zmiana znaku y_p jest spowodowana, tym że oś $+Y$ we współrzędnych ekranowych skierowana jest w dół.